

Sesión 10 — El juego decide solo

Godot 4.7 · GDScript. Todo lo de hoy es **la misma idea**: un array recorrido con un índice.

Lo nuevo de hoy, en una tabla

Qué	Para qué
<code>enum Estado { A, B }</code>	Lista de nombres legibles en vez de números sueltos. <code>Estado.keys()[i]</code> devuelve el nombre como texto, útil para imprimir.
<code>match valor:</code>	El <code>switch</code> de GDScript. Más legible que una cadena larga de <code>if/elif</code> cuando las ramas son fijas.
<code>i = (i + 1) % n</code>	Avanzar un índice en bucle: 0,1,2,...,n-1,0,1,... Nunca se sale del array.
<code>randi_range(0, n - 1)</code>	Índice entero al azar dentro de un array. El <code>- 1</code> es obligatorio: con <code>size()</code> se sale.
<code>randf_range(a, b)</code>	Número decimal al azar entre a y b. Para posiciones, tiempos, tamaños.
<code>get_first_node_in_group("x")</code>	Encontrar un nodo por etiqueta en vez de por ruta. No se rompe si alguien mueve nodos de sitio.
<code>RayCast3D</code>	Rayo invisible entre dos puntos: sirve como línea de visión.

Antes de nada: el grupo "player"

Seleccionar el personaje > panel **Node** > pestaña **Groups** > escribir **player** > **Add**.

Si se olvida esto, el enemigo compila pero se queda quieto para siempre. Es el fallo número uno de hoy.

Receta A — Enemigo con máquina de estados

Árbol: `CharacterBody3D "Enemigo"` > `MeshInstance3D (CapsuleMesh, Y 0.875)` + `CollisionShape3D (CapsuleShape3D, Y 0.875)` + `RayCast3D` renombrado **Vision** (Y 0.9).

```
extends CharacterBody3D

enum Estado { PATRULLAR, PERSEGUIR, ATACAR, HUIR }

@export var velocidad: float = 3.0
```

```

@export var aceleracion: float = 10.0
@export var rango_vision: float = 8.0
@export var rango_ataque: float = 1.5
@export var vida: float = 100.0
@export var vida_huida: float = 30.0
@export var altura_ojos: float = 0.9
# Desplazamientos desde donde se colocó al enemigo, no posiciones absolutas.
@export var ruta: Array[Vector3] = [Vector3(0, 0, 0), Vector3(8, 0, 0)]

var estado_actual: Estado = Estado.PATRULLAR
var indice_punto: int = 0
var jugador: Node3D = null
var puntos: Array[Vector3] = []

@onready var vision: RayCast3D = $Vision

func _ready() -> void:
    jugador = get_tree().get_first_node_in_group("player")
    for desplazamiento in ruta:
        puntos.append(global_position + desplazamiento)

func _physics_process(delta: float) -> void:
    velocity += get_gravity() * delta

    # 1. MEDIR
    var distancia := INF
    if jugador != null:
        distancia = _distancia_plana(global_position, jugador.global_position)
    var lo_veo := _tiene_linea_vision()

    # 2. DECIDIR
    var nuevo := _decidir_estado(distancia, lo_veo)
    if nuevo != estado_actual:
        # imprimir solo al CAMBIAR
        estado_actual = nuevo
        print("Enemigo -> ", Estado.keys()[estado_actual])

    # 3. ACTUAR
    match estado_actual:
        Estado.PATRULLAR: _patrullar(delta)
        Estado.PERSEGUIR: _moverse_hacia(jugador.global_position, 1.0, delta)
        Estado.ATACAR: _frenar(delta)
        Estado.HUIR: _moverse_lejos_de(jugador.global_position, delta)

    move_and_slide()

# EL ORDEN ES LA LÓGICA: el primer if que se cumple gana.

```

```

func _decidir_estado(distancia: float, lo_veo: bool) -> Estado:
    if vida <= vida_huida and distancia <= rango_vision:
        return Estado.HUIR
    if distancia <= rango_ataque and lo_veo:
        return Estado.ATACAR
    if distancia <= rango_vision and lo_veo:
        return Estado.PERSEGUIR
    return Estado.PATRULLAR

func _tiene_linea_vision() -> bool:
    if jugador == null:
        return false
    # Apuntar al PECHO, no al origen (que está en los pies, a ras del suelo).
    var objetivo: Vector3 = jugador.global_position + Vector3(0, altura_ojos, 0)
    vision.target_position = vision.to_local(objetivo)    # LOCAL, no global
    vision.force_raycast_update()
    if vision.is_colliding():
        return vision.get_collider() == jugador    # ¿era él o una pared?
    return true

func _patrullar(delta: float) -> void:
    if puntos.is_empty():
        _frenar(delta); return
    var destino: Vector3 = puntos[indice_punto]
    _moverse_hacia(destino, 1.0, delta)
    if _distancia_plana(global_position, destino) < 0.5:
        indice_punto = (indice_punto + 1) % puntos.size()    # ciclar

func _moverse_hacia(destino: Vector3, factor: float, delta: float) -> void:
    var direccion := destino - global_position
    direccion.y = 0.0
    var objetivo := Vector3.ZERO
    if direccion.length() > 0.05:
        objetivo = direccion.normalized() * velocidad * factor
    velocity.x = move_toward(velocity.x, objetivo.x, aceleracion * delta)
    velocity.z = move_toward(velocity.z, objetivo.z, aceleracion * delta)
    if objetivo.length() > 0.1:
        basis = basis.slerp(Basis.looking_at(objetivo), delta * 8.0).orthonormalized()

func _moverse_lejos_de(amenaza: Vector3, delta: float) -> void:
    # Huir = perseguir con la dirección invertida. Nada más cambia.
    var direccion := global_position - amenaza
    direccion.y = 0.0
    _moverse_hacia(global_position + direccion, 1.3, delta)

func _frenar(delta: float) -> void:

```

```

velocity.x = move_toward(velocity.x, 0.0, aceleracion * delta)
velocity.z = move_toward(velocity.z, 0.0, aceleracion * delta)

# Distancia ignorando la altura: si no, un jugador saltando "se aleja".
func _distancia_plana(a: Vector3, b: Vector3) -> float:
    return Vector2(a.x - b.x, a.z - b.z).length()

```

No aparece ningún enemigo en la escena: creaste `Enemigo.tscn` pero no lo instanciaste en `main.tscn` (Instantiate Child Scene).

La consola queda muda, sin ningún Enemigo -> ...: falta el grupo `player`, o el `RayCast3D` no se llama exactamente `Vision` — `$Vision` queda en `null`.

El enemigo no se mueve: falta el grupo `player` en el personaje.

El enemigo nunca ataca, solo persigue: tienes `PERSEGUIR` antes que `ATACAR` en `_decidir_estado`.

El enemigo ve a través de las paredes: falta comparar `get_collider() == jugador`.

La consola se inunda: estás imprimiendo cada frame, no solo al cambiar de estado.

El enemigo persigue de espaldas: le pusiste un modelo de Mixamo — `Rotation Y = 180` en el nodo del modelo, igual que al jugador en la Sesión 9.

Receta B — Rival CPU por waypoints (Mario Kart)

Es el mismo kart de la Sesión 9. Solo cambia de dónde salen el acelerador y el volante: de la ruta, no del Input.

```

extends CharacterBody3D

@export var ruta: Array[Vector3] = [
    Vector3(0, 0, -20), Vector3(20, 0, -20),
    Vector3(20, 0, 0), Vector3(0, 0, 0),
]
@export var distancia_llegada: float = 4.0 # muy chico = se queda dando vueltas
@export var velocidad_maxima: float = 12.0
@export var aceleracion: float = 8.0
@export var velocidad_giro: float = 2.2

var velocidad_actual: float = 0.0
var indice_punto: int = 0

func _physics_process(delta: float) -> void:
    if ruta.is_empty():
        return

    var destino: Vector3 = ruta[indice_punto]
    if _distancia_plana(global_position, destino) < distancia_llegada:
        indice_punto = (indice_punto + 1) % ruta.size() # mismo % de antes
        destino = ruta[indice_punto]

```

```

var hacia := destino - global_position
hacia.y = 0.0
# Pasar el vector a coordenadas DEL KART: x positivo = está a mi derecha.
var local := transform.basis.inverse() * hacia
var volante: float = clampf(-local.x, -1.0, 1.0)

# --- de aquí abajo, idéntico al kart del jugador ---
velocidad_actual = move_toward(velocidad_actual, velocidad_maxima, aceleracion * delta)
var proporcion: float = abs(velocidad_actual) / velocidad_maxima
if proporcion > 0.05:
    rotate_y(volante * velocidad_giro * proporcion * signf(velocidad_actual) * delta)
if not is_on_floor():
    velocity += get_gravity() * delta
var adelante := -transform.basis.z * velocidad_actual
velocity.x = adelante.x
velocity.z = adelante.z
move_and_slide()

func _distancia_plana(a: Vector3, b: Vector3) -> float:
    return Vector2(a.x - b.x, a.z - b.z).length()

```

RECETA C — Generar el nivel por código

Esta receta va sin resolver, a propósito. Aquí tienes la estructura, los `@export` y una función ya hecha. **El algoritmo lo escribes tú.** Las recetas A y B llevan el código completo porque la A se escribió entre todos en clase y la B es tu propio kart de la Sesión 9 con otro origen de datos; esta es trabajo nuevo.

Lo que las tres variantes comparten

- Un **array de escenas** con los tipos de pieza. Agregar un tipo más basta para que empiece a aparecer, sin tocar otra línea.
- `randi_range(0, n - 1)` para elegir una al azar. El `- 1` es obligatorio.
- `instantiate() + add_child(nodo, true) + posicionar`. El `true` pide nombres legibles en la consola.
- Una lista de **piezas vivas**, para poder quitarlas después.

La trampa: ¿cuál se salta este bucle?

```

# La tarea: sacar de la lista todas las piezas rotas.
for i in range(vivos.size()):
    if vivos[i].esta_rota:
        vivos.remove_at(i)

```

Al borrar el elemento `i`, los siguientes **se corren un puesto**. El bucle avanza a `i + 1`, que ahora tiene lo que estaba en `i + 2`: **se salta el que quedó en su lugar**. Falla justo cuando hay piezas rotas **seguidas**. Con 10 piezas de las que 6 estaban rotas, este bucle deja **2 sin borrar**.

La forma correcta la escribes tú: recorrer del final al principio, porque lo que se corre ya quedó revisado.

El esqueleto

```
extends Node3D

@export var tipos: Array[PackedScene] = []
# ... más @export según tu variante ...

var vivos: Array[Node3D] = []      # las piezas que existen ahora mismo

# --- ESTO LO ESCRIBES TÚ -----
## Decide DÓNDE va cada pieza y llama a _crear() una vez por cada una.
func _generar() -> void:
    pass

# --- ESTO LO ESCRIBES TÚ -----
## Saca una pieza de la lista de vivos. Ojo con el sentido del recorrido.
func _quitar(pieza: Node3D) -> void:
    pass

# --- ESTO YA VIENE HECHO -----
func _crear(escena: PackedScene, posicion: Vector3) -> void:
    var pieza := escena.instantiate() as Node3D
    add_child(pieza, true)
    pieza.global_position = posicion
    vivos.append(pieza)
```

C1 · Subway Surfers — obstáculos infinitos

Qué tiene que lograr: la pista se inventa **por delante** del corredor y se borra **por detrás**. Sin lo segundo el juego acumula miles de nodos y se arrastra.

Escenas: dos obstáculos con raíz **Area3D**: uno bajo (BoxMesh $2 \times 0.8 \times 0.8$, Y 0.4) que se salta, y uno alto ($2 \times 2.5 \times 0.8$, Y 1.25) que obliga a cambiar de carril. Más un Node3D "Generador".

Tus @export: tipos.carriles: Array[float] = [-3.0, 0.0, 3.0] (los mismos del corredor) · jugador: Node3D · distancia_adelante = 60.0 · separacion = 12.0 · distancia_borrado = 15.0 · var proximo_z: float = -25.0

Lo que escribes: generar mientras el horizonte quede por delante del jugador, con carril y tipo al azar; y borrar lo que quedó atrás.

Pista: tiene que ser un while, no un if — si el corredor va rápido puede hacer falta crear varios obstáculos en un mismo frame, o queda un hueco en la pista.

Cómo sabes que funciona: tras varios minutos el número de piezas vivas se queda **estable** (unas 6) en vez de crecer. Si

crece, no se están borrando.

C2 · Arkanoid — la rejilla de ladrillos

Qué tiene que lograr: construir el muro al empezar el nivel, **centrado** respecto al origen, y sacar cada ladrillo de la lista cuando se rompa.

Escenas: un **StaticBody3D** "Ladrillo" (BoxMesh $1.1 \times 0.45 \times 0.5$ + CollisionShape3D igual) con un script corto: `@export var resistencia: int`, una señal `destruido(ladrillo)`, y un `golpear()` que resta resistencia y, al llegar a cero, emite y hace `queue_free()`. Haz **dos versiones** con distinto color y resistencia, para que el array de tipos sirva de algo. Más un **Node3D** "Muro".

Tus @export: `tipos · filas: int = 5 · columnas: int = 8 · separacion: Vector2 = Vector2(1.3, 0.6) · altura_base: float = 3.0`

Lo que escribes: dos bucles anidados —filas dentro de columnas— y la fórmula que convierte (`fila`, `columna`) en una posición.

Lo difícil no es el bucle, es el centrado. Si escribes `x = columna * separacion` el muro te arranca en el origen y crece solo hacia la derecha. La pregunta es: *¿cuánto hay que correr todo hacia la izquierda para que quede centrado?*

Cómo sabes que funciona: tienen que salir exactamente **filas × columnas** ladrillos (con esos valores, 40); el de más a la izquierda y el de más a la derecha quedan a la **misma distancia** del origen; y al romper 6, la lista queda en 34 — ni 32 ni 35.

Reto: dejar huecos — que algunas casillas queden vacías según una probabilidad, con `randf() < 0.15` y un `continue`.

C3 · AimLabs — la aparición de targets

Qué tiene que lograr: mantener siempre N targets en pantalla, en posiciones al azar dentro de una zona; al acertarle a uno, desaparece y aparece otro.

Escenas: un **Area3D** "Target" (SphereMesh + SphereShape3D) con un script de tres líneas: una señal `acertado(target)` y un `acertar()` que la emite y hace `queue_free()`. Más un **Node3D** "Generador".

Tus @export: `tipos · cuantos_a_la_vez: int = 3 · zona: Vector3 = Vector3(10, 5, 2)` (ancho, alto, profundidad de la caja donde pueden salir) · `centro: Vector3`

Lo que escribes: una función que devuelve un punto al azar **dentro** de esa caja, y la reacción al acierto: quitarlo de la lista y hacer aparecer otro.

Pista: `randf_range` aplicado eje por eje, media zona hacia cada lado del centro.

Cómo sabes que funciona: tras acertarle a muchos seguidos, la cantidad de targets vivos **no cambia nunca**. Y ninguno aparece fuera del volumen: si cambias ZONA en el Inspector con el juego corriendo, el área donde salen tiene que cambiar con ella.

Reto: que un target nuevo no aparezca demasiado cerca del anterior — repetir el sorteo hasta que la distancia sea suficiente.

¿Cuerpo sólido o área? La regla: **sólido si algo tiene que chocar de verdad, área si solo tiene que avisar.**

Subway Surfers → área. Con cuerpos sólidos el corredor se queda trabado corriendo en el sitio.

Arkanoid → sólido. La pelota tiene que **rebotar** contra el ladrillo.

AimLabs → área. El target no frena nada, solo avisa que le acertaste.

Y avisar es la señal `body_entered` de la Sesión 5.